UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, MANAGUA
UNAN-MANAGUA

Centro Universitario Regional de Chontales
“Cornelio Silva Argüello”
UNAN - CUR - Chontales

OPTATIVA I: DESARROLLO MÓVIL HÍBRIDO

Tema

Tema I: Fundamentos de Desarrollo Móvil Híbrido.

Carrera

Ingeniería en Sistemas de Información – 4to año.

Fecha

Miércoles 8 de Abril de 2026.

Periodo

1:00 p. m. – 2:30 p. m.

Docente

Eliab Javier Selva Cruz

Objetivo de aprendizaje

- Desarrollar aplicaciones móviles híbridas desde cero, comprendiendo su estructura y funcionamiento.

Configuraciones e inserción de datos

1. Asegúrese de importar **bootstrap.min.css** y **bootstrap-icons.css** en el archivo **main.js**:

```
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap-icons/font/bootstrap-icons.css';
```

2. Asegúrese de definir las siguientes sentencias CSS en su archivo **App.css**:

```
.margen-superior-main {
  margin-top: 50px;
  padding: 20px;
}

.color-navbar {
  background-color: #1068db !important;
  background-image: linear-gradient(90deg, #121b96 0%, #2889b6 50%, #2697cc 100%);
}

.color-texto-marca {
  color: #1e3d87 !important;
  font-size: 18px;
}
```

3. Codifique las siguientes sentencias dentro del archivo **RutaProtegida.jsx**:

```
import React from "react";
import { Navigate } from "react-router-dom";

const RutaProtegida = ({ children }) => {
  // Verifica si el usuario está autenticado usando localStorage
  const estaLogueado = !!localStorage.getItem("usuario-supabase");

  // Log para depuración
  console.log("Usuario autenticado:", estaLogueado);

  // Si está autenticado,redirige a la página de login
  return estaLogueado ? children : <Navigate to="/login" replace />;
};

export default RutaProtegida;
```

4. Codifique le siguiente componente en el archivo **NotificacionOperacion.jsx**:

```jsx
import { useEffect, useState } from 'react';
import { Toast, ToastContainer } from 'react-bootstrap';

const NotificacionOperacion = ({ mostrar, mensaje, tipo, onCerrar }) =>{
  const [visible, setVisible] = useState(mostrar);

  useEffect(() => {
    setVisible(mostrar);
  }, [mostrar]);

  const fechaLocal = () => {
    const f = new Date();
    const fecha = new Date(f);
    const anio = fecha.getFullYear();
    const mes = String(fecha.getMonth() + 1).padStart(2, '0');
    const dia = String(fecha.getDate()).padStart(2, '0');
    return `${dia}-${mes}-${anio} ${fecha.toTimeString().slice(0, 5)}`;
  }

  return (
    <ToastContainer position="top-center" className="p-2">
      <Toast
        onClose={() => {
          setVisible(false);
          onCerrar();
        }}
        show={visible}
        delay={2500}
        autohide
        bg={tipo === 'exito' ? 'success' : tipo === 'advertencia' ? 'warning' : 'danger'}
      >
        <Toast.Header>
          <strong className="me-auto">{tipo === 'exito' ? '✅ Éxito' : tipo === 'advertencia' ? '⚠ Advertencia' : '❌ Error'}</strong>
          <small>{fechaLocal()}</small>
        </Toast.Header>
        <Toast.Body className={tipo === 'exito' || tipo === 'error' ? 'text-white' : ''}>
          {mensaje}
        </Toast.Body>
      </Toast>
    </ToastContainer>
  );
}

export default NotificacionOperacion;
```

Este componente es una **notificación emergente temporal (Toast)** que utiliza **React-Bootstrap** para informar al usuario sobre el resultado de una acción (éxito, advertencia o error).

Funcionamiento:

- **Gestión de visibilidad:** Utiliza un estado interno (visible) sincronizado mediante useEffect con la prop mostrar para aparecer o desaparecer.
- **Auto-ocultado:** La notificación se cierra automáticamente después de **2.5 segundos** (delay={2500}).
- **Adaptación visual:** Cambia dinámicamente el color de fondo (bg) y el icono del encabezado según la prop tipo (exito, advertencia o error).
- **Información contextual:** Muestra un mensaje personalizado y genera automáticamente la **fecha y hora actual** formateada (DD-MM-AAAA HH:mm).
- **Limpieza:** Al cerrarse, ejecuta la función onCerrar() pasada por props, permitiendo que el componente padre resetee el estado de la notificación.

5. Dentro del archivo **ModalRegistroCategoria.jsx**, defina las siguientes importaciones, función, props y exportación por defecto:

```
import React, {useState} from "react";
import { Modal, Form, Button } from "react-bootstrap";

const ModalRegistroCategoria = ({
  mostrarModal,
  setMostrarModal,
  nuevaCategoria,
  manejoCambioInput,
  agregarCategoria,
}) => {

};

export default ModalRegistroCategoria;
```

6. Dentro de la función del componente, defina la siguiente variable de estado y método para manejar el registro de la categoría:

```
const [deshabilitado, setDeshabilitado] = useState(false);

const handleRegistrar = async () => {
  if (deshabilitado) return;
  setDeshabilitado(true);
  await agregarCategoria();
  setDeshabilitado(false);
};
```

7. Dentro del cuerpo del componente, retorne los siguientes elementos visuales:

```
return (
  <Modal
    show={mostrarModal}
    onHide={() => setMostrarModal(false)}
    backdrop="static"
    keyboard={false}
    centered
  >
    <Modal.Header closeButton>
      <Modal.Title>Agregar Categoría</Modal.Title>
    </Modal.Header>
    <Modal.Body>
      <Form>
        <Form.Group className="mb-3">
          <Form.Label>Nombre</Form.Label>
          <Form.Control
            type="text"
            name="nombre_categoria"
            value={nuevaCategoria.nombre_categoria}
            onChange={manejoCambioInput}
            placeholder="Ingresa el nombre"
          />
        </Form.Group>
        <Form.Group className="mb-3">
          <Form.Label>Descripción</Form.Label>
          <Form.Control
            as="textarea"
            rows={3}
            name="descripcion_categoria"
            value={nuevaCategoria.descripcion_categoria}
            onChange={manejoCambioInput}
            placeholder="Ingresa la descripción"
          />
        </Form.Group>
      </Form>
    </Modal.Body>
    <Modal.Footer>
      <Button variant="secondary" onClick={() => setMostrarModal(false)}>
        Cancelar
      </Button>
      <Button
        variant="primary"
        onClick={handleRegistrar}
        disabled={nuevaCategoria.nombre_categoria.trim() === "" || deshabilitado}
      >
        Guardar
      </Button>
    </Modal.Footer>
  </Modal>
);
```

8. Dentro del archivo **Categorias.jsx**, defina las siguientes importaciones, función, props y exportación por defecto:

```
import React, { useState } from "react";
import { Container, Row, Col, Button } from "react-bootstrap";
import { supabase } from "../database/supabaseconfig";

import ModalRegistroCategoria from "../components/categorias/ModalRegistroCategoria";
import NotificacionOperacion from "../components/NotificacionOperacion";

const Categorias = () => {

  return (
    <Container className="mt-3">

    </Container>
  );
};

export default Categorias;
```

9. Dentro de la función, defina las siguientes variables de estado y método para el manejo de cambio de valores en los inputs del formulario de registro.

```
const [toast, setToast] = useState({ mostrar: false, mensaje: "", tipo: "" });
const [mostrarModal, setMostrarModal] = useState(false);

const [nuevaCategoria, setNuevaCategoria] = useState({
  nombre_categoria: "",
  descripcion_categoria: "",
});

const manejoCambioInput = (e) => {
  const { name, value } = e.target;
  setNuevaCategoria((prev) => ({
    ...prev,
    [name]: value,
  }));
};
```

Para insertar datos en **Supabase** utilizando su cliente de JavaScript/TypeScript, se utiliza una sintaxis muy declarativa basada en promesas.

La sintaxis estándar es:

```
const { data, error } = await supabase
  .from('nombre_de_la_tabla')
  .insert([
    { columna1: 'valor1', columna2: 'valor2' }
  ])
  .select(); //.select() es opcional, para retornar la fila creada
```

- **.from('tabla')**: Especifica a qué tabla de tu base de datos de Postgres quieres enviar la información.

- **.insert([...])**:
  - Recibe un **array de objetos**.
  - Cada objeto representa una fila.

- Las llaves del objeto deben coincidir exactamente con los nombres de las columnas en Supabase.

- **Inserción Múltiple**: Puedes insertar varias filas a la vez simplemente agregando más objetos al array: .insert([{...}, {...}, {...}]).

- **Manejo de Errores**: Supabase siempre devuelve un objeto con { data, error }. Es fundamental validar si error existe antes de continuar.

10. Defina el siguiente método para validar y registrar la categoría en supabase:

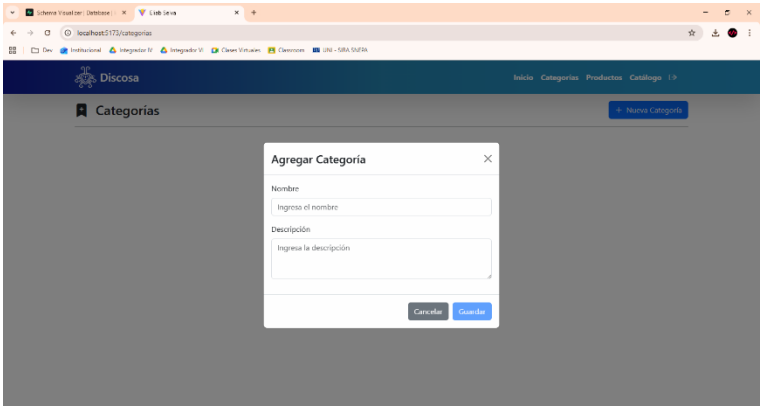
```
const agregarCategoria = async () => {
  try {
    if (
      !nuevaCategoria.nombre_categoria.trim() ||
      !nuevaCategoria.descripcion_categoria.trim()
    ) {
      setToast({
        mostrar: true,
        mensaje: "Debe llenar todos los campos.",
        tipo: "advertencia",
      });
      return;
    }

    const { error } = await supabase.from("categorias").insert([
      {
        nombre_categoria: nuevaCategoria.nombre_categoria,
        descripcion_categoria: nuevaCategoria.descripcion_categoria,
      },
    ]);

    if (error) {
      console.error("Error al agregar categoría:", error.message);
      setToast({
        mostrar: true,
        mensaje: "Error al registrar categoría.",
        tipo: "error",
      });
      return;
    }

    // Éxito
    setToast({
      mostrar: true,
      mensaje: `Categoría "${nuevaCategoria.nombre_categoria}" registrada exitosamente.`,
      tipo: "exito",
    });

    // Limpiar formulario y cerrar modal
    setNuevaCategoria({ nombre_categoria: "", descripcion_categoria: "" });
    setMostrarModal(false);

  } catch (err) {
    console.error("Excepción al agregar categoría:", err.message);
    setToast({
      mostrar: true,
      mensaje: "Error inesperado al registrar categoría.",
      tipo: "error",
    });
  }
};
```

11. Dentro del componente bootstrap **Container** (en el return), defina las siguientes sentencias para el título, botón e implementación de componentes:

```
{/* Título y botón Nueva Categoría */}
<Row className="align-items-center mb-3">
  <Col xs={9} sm={7} md={7} lg={7} className="d-flex align-items-center">
    <h3 className="mb-0">
      <i className="bi-bookmark-plus-fill me-2"></i> Categorías
    </h3>
  </Col>
  <Col xs={3} sm={5} md={5} lg={5} className="text-end">
    <Button
      onClick={() => setMostrarModal(true)}
      size="md"
    >
      <i className="bi-plus-lg"></i>
      <span className="d-none d-sm-inline ms-2">Nueva Categoría</span>
    </Button>
  </Col>
</Row>

<hr />

{/* Modal de Registro */}
<ModalRegistroCategoria
  mostrarModal={mostrarModal}
  setMostrarModal={setMostrarModal}
  nuevaCategoria={nuevaCategoria}
  manejoCambioInput={manejoCambioInput}
  agregarCategoria={agregarCategoria}
/>

{/* Notificación */}
<NotificacionOperacion
  mostrar={toast.mostrar}
  mensaje={toast.mensaje}
  tipo={toast.tipo}
  onCerrar={() => setToast({ ...toast, mostrar: false })}
/>
```

12. Asegúrese de crear su tabla categorias dentro de supabase, dejando la clave primaria como identity y definiciendo los siguientes atributos:

13. Finalmente, realice pruebas de inserción y verifique los registros en el editor de tablas de supabase:



Actividades

I. Asignación individual:

a. En el espacio designado en Classroom entregue lo siguiente:

   i. Enlace de confirmación con los cambios realizados en el repositorio de GitHub de la práctica de esta guía.
   ii. Enlace de Netlify con los cambios aplicados en la aplicación desplegada.

II. Asignación grupal:

a. Configure proyecto Vite grupal siguiendo las configuraciones practicadas.
b. Entregue enlace de repositorio grupal de GitHub con el código del proyecto. Agregue al docente (eliabjselvacruz51@gmail.com) como colaborador del proyecto grupal. Todos los integrantes del grupo deben colaborar (realizar confirmaciones) con el proyecto.
c. Cree usuario de acceso para el docente dev51unan@gmail.com y con clave 123456.
d. Entregue enlace de Netlify con el proyecto grupal desplegado.